동료 연구자를 위한 실전 안내서

AI와 함께 논문 쓰고 발표하기

국민건강영양조사 데이터를 쓰는 의사 연구자를 위한,
한 폴더 · 일곱 스킬 · 여덟 단계 워크플로

OpenCode | 국민건강영양조사 | IMRaD | 학회 발표 | 재현 가능

만든 사람
김현수 · 가정의학과

사례
HCC vs 혈관종, B-mode 초음파

읽는 시간
약 45분 (실습 포함 2시간)

버전
v1.0 · 2026-10-25

CONTENTS

이 안내서의 순서

처음이면 1→2→3만 읽고 바로 실습한다. 나머지는 필요할 때 펼친다.

00 왜 이 가이드인가
상을 받은 과정을 그대로 옮겼습니다

01 30분 안에 첫 성공
설치 · 스킬 · 첫 대화

02 챗봇과 에이전트는 다릅니다
작동 원리를 한 장으로

03 작업 폴더 만들기
AI가 읽고 쓸 책상

04 일곱 개의 스킬
언제 무엇을 부르나

05 여덟 단계 워크플로
질문에서 큐카드까지

06 국민건강영양조사 특별편
복합표본설계가 절반입니다

07 실제 사례: HCC vs 혈관종
숫자와 실패까지

07.5 상을 목표로 한다면
심사위원이 실제로 묻는 세 가지

08 대화 규칙 열둘
hand-to-hand로 일하는 법

09 믿지 않을 것은 믿지 않기
환각 · 인용 · 개인정보 · AI 공개

10 복사해서 쓰는 프롬프트
15개

11 8주 로드맵과 체크리스트
오늘 저녁에 할 일

00 / WHY

상을 받은 것은 결과지만,
여러분께 드릴 것은 과정입니다

2026년 7월부터 10월까지, 한 저장소에 남은 기록.

논문은 연구의 절반이고, 발표는 나머지 절반입니다. 그런데 둘 다 만드는 데는 연구보다 더 오랜 시간이 걸립니다. 저는 그 시간을 AI에게 통째로 맡기지 않고, **일을 나눠서 맡겼습니다.**

2026년 가을, 저는 간암(HCC)과 혈관종을 B-mode 초음파에서 구분하는 연구를 대한가정의학회 추계학술대회에서 발표했습니다. 8분 발표, 2분 질의. 발표자료 13장, 대본 7분 35초, 큐카드 19장. 그리고 그 과정에서 상을 받았습니다.

이 안내서는 그 상을 자랑하려고 쓴 것이 아닙니다. **다음 사람이 같은 시간을 덜 쓰게 하려고** 썼습니다. 제가 실제로 쓴 폴더 구조, 프롬프트, 검증 순서, 실패한 지점을 그대로 남겼습니다.

여러분의 연구는 제 연구와 다릅니다. 대부분 이미지가 아니라 **국민건강영양조사(KNHANES) 원시자료를** 씁니다. 그래서 이 안내서는 이미지 연구 사례를 예로 들되, **표 형식 데이터에 그대로 적용되는 방식**으로 다시 썼습니다.

이 가이드가 약속하는 것

설치부터 첫 논문 초안, 학회 발표 큐카드까지
손을 잡고 한 단계씩

갑니다. AI를 잘 몰라도 됩니다. 명령어를 외울 필요도 없습니다. 복사해서 붙여넣을 문장을 준비했습니다.

약속하지 않는 것
AI가 통계를 대신 책임지지 않습니다. 인용을 지어내지 않는다는 보장도 없습니다. **마지막 판단과 서명은 사람이 합니다.** 이 가이드는 그 판단을 빠르게 만드는 도구입니다.

> “AI에게 논문을 써 달라고 하지 마세요. **AI에게 연구를 검증하게** 하세요. 순서가 반대입니다.”
>
> 이 가이드의 한 줄 원칙

01 / FIRST RUN

30분 안에 첫 성공을 만듭니다

목표: AI가 내 폴더의 파일 하나를 읽고, 한 문단을 고쳐 쓰게 한다.

설치는 세 단계입니다. ① OpenCode를 깐다 ② 이 저장소의 스킬을 복사한다 ③ 폴더를 열고 한 문장을 보낸다. 아래는 Windows 11 기준이며, macOS는 같은 순서에 명령만 다릅니다.

1 OpenCode를 설치합니다

터미널이 낯설면 **Desktop** 앱을 쓰세요. 화면은 일반 채팅 앱과 비슷합니다.

OpenCode는 채팅창이 아니라 **내 컴퓨터의 폴더에서 직접 일하는 에이전트**입니다. 설치 파일은 공식 배포 페이지에서 받습니다.

- **Windows 설치 파일** — opencode-desktop-win-x64.exe (https://opencode.ai/update/api/latest/desktop/opencode/opencode-desktop-win-x64.exe) (ARM 노트북은 arm64 (https://opencode.ai/update/api/latest/desktop/opencode/opencode-desktop-win-arm64.exe))
- **macOS** — Apple silicon (https://opencode.ai/update/api/latest/desktop/opencode/opencode-desktop-mac-arm64.dmg) · Intel (https://opencode.ai/update/api/latest/desktop/opencode/opencode-desktop-mac-x64.dmg)
- 공식 문서: opencode.ai/v2/docs (https://opencode.ai/v2/docs/)

설치 후 앱을 열고 **Connect**(또는 설정 → Providers)에서 모델을 연결합니다. 처음이면 **OpenCode Go** 구독(월 10달러 수준)이 키 관리 없이 가장 간단합니다. DeepSeek 같은 API 키를 직접 넣어도 됩니다.

계정과 결제는 제가 대신 못 합니다

연구비 카드로 결제할지, 개인 카드로 할지는 소속 기관 규정을 먼저 확인하세요. 결제 정보는 절대 채팅창에 붙여넣지 않습니다.

2 이 저장소를 받고, 스킬을 복사합니다

Git을 몰라도 됩니다. GitHub에서 **Code → Download ZIP**을 누르면 끝입니다.

압축을 풀면 `ai-research-copilot` 폴더가 생깁니다. 그 안의 `skills` 폴더를 OpenCode가 읽는 위치로 복사합니다.

- **Windows**: `C:\Users\<내 이름>\.config\opencode\skills`
- **macOS·Linux**: `~/.config/opencode/skills`

탐색기에서 `.config`가 안 보이면 폴더 주소창에 경로를 직접 입력하면 됩니다. 또는 저장소의 설치 스크립트를 실행하세요.

PROMPT

```
# Windows PowerShell — 저장소 폴더에서 실행
powershell -ExecutionPolicy Bypass -File .\install\install-windows.ps1
```

스크립트는 `skills` 폴더를 위 경로로 복사하고, 이미 있으면 덮어쓰기 전에 물어봅니다. 관리자 권한은 필요 없습니다.

> **스킬이 보이는지 확인하는 법**
> OpenCode를 다시 켠 뒤 대화창에 `@academic-research-writing`을 입력했을 때 자동완성 목록에 뜨면 성공입니다. 안 뜨면 11장의 "스킬이 안 보일 때"를 보세요.

---

3 작업 폴더를 열고, 한 문장을 보냅니다

AI에게 "내 폴더를 보라"고 말로만 하지 말고, 폴더 자체를 열어 줍니다.

바탕화면에 `my-paper` 폴더를 만들고, 아무 문서나 하나 넣습니다. OpenCode Desktop에서 **Open Folder**로 그 폴더를 엽니다. 그러면 AI의 현재 위치가 그 폴더가 됩니다.

PROMPT

```
이 폴더에 있는 파일이 뭐가 있는지 알려줘.
아직 아무것도 고치지 말고, 읽기만 해.
```

AI가 파일 목록과 각 파일의 역할을 요약하면 첫 성공입니다. 이제 같은 폴더에서 아래를 보냅니다.

PROMPT

```
@academic-research-writing
이 폴더의 초록 초안을 읽고, 연구 질문(PICO)이 한 문장으로
명확히 드러나는지 검토해줘. 고치지는 말고, 문제 3개만 짚어줘.
```

답이 오면 **“고쳐줘”가 아니라 “짚어줘”**부터 했다는 점을 기억하세요. 이 순서가 이 가이드 전체의 리듬입니다.

여기서 막히면

① 앱이 안 열린다 → 재부팅 후 다시 실행. ② 모델 연결이 안 된다 → 결제·키 문제이므로 `/connect` 화면을 다시 확인. ③ 스킬이 안 보인다 → 복사 경로의 철자 확인 (`.config/opencode/skills/<스킬이름>/SKILL.md`). ④ 그래도 안 되면 11장 FAQ로.

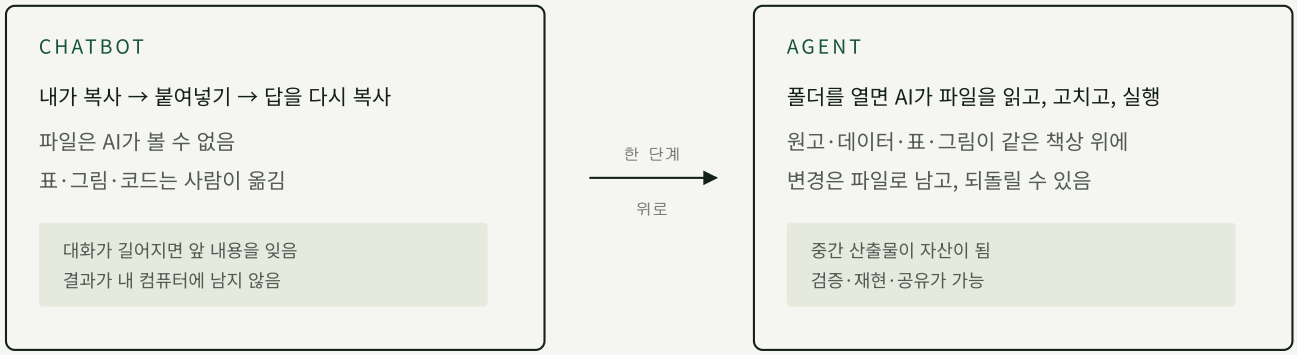
---

02 / MENTAL MODEL

챗봇은 말하고,
에이전트는 파일을 고칩니다

이 차이를 모르면 첫 주에 반드시 사고가 납니다.

ChatGPT 창에 논문을 붙여넣고 “다듬어 줘”라고 하는 것과, OpenCode에게 “이 폴더의 원고를 읽고 표 2와 본문 숫자가 맞는지 확인해”라고 하는 것은 **다른 일**입니다.



챗봇은 대화가 결과물이고, 에이전트는 폴더가 결과물입니다. 그래서 폴더 구조가 곧 연구의 설계도가 됩니다.

에이전트가 하는 네 가지

- **읽기** — 폴더의 원고·CSV·코드·PDF를 읽습니다.
- **쓰기** — 새 파일을 만들거나 기존 파일을 고칩니다.
- **실행** — Python·R 코드를 돌려 표와 그림을 만듭니다.
- **검색** — 웹과 논문 데이터베이스에서 근거를 찾습니다.

이 네 가지를 하려면 **경계가** 필요합니다. 그래서 OpenCode는 폴더를 열게 하고, 위험한 작업은 허가를 받게 합니다. “파일을 지워도 됩니까?”라고 물으면 대부분은 **아니오**입니다.

기억해야 할 세 문장

01

폴더 밖은 건드리지 않습니다

작업 폴더를 논문 하나로 좁히면 사고 범위도 좁아집니다.

02

대화는 사라지고 파일은 남습니다

중요한 결정은 채팅이 아니라 `notes/decision.md`에 적게 하세요.

03

한 번에 한 가지

“논문 전체를 써줘”는 실패합니다. “표 1의 변수 정의를 맞춰줘”가 성공합니다.

03 / WORKSPACE

AI가 읽고 쓸 책상을 먼저 만듭니다

폴더 이름은 영어, 내용은 한국어. 이 조합이 가장 덜 깨집니다.

논문 한 편 = 폴더 하나. 이 원칙만 지키면 나중에 파일을 찾느라 반나절을 쓰지 않습니다. 아래 구조를 그대로 만들어도 되고, 필요한 것만 골라도 됩니다.

PROMPT

```
# my-paper/ - 논문 한 편의 책상
my-paper/
├── 00_admin/          IRB, 저널 규정, 이해상충
├── 01_data/
│   ├── raw/           원본 - 절대 수정하지 않음
│   ├── derived/       코드가 만든 분석용 데이터
│   └── codebook/      변수 설명서, 설문지
├── 02_analysis/
│   ├── 01_clean.R     정제 · 결측 · 가중치
│   ├── 02_table1.R    표 1
│   ├── 03_models.R    주 분석
│   └── logs/          실행 로그
├── 03_output/
│   ├── tables/        CSV + docx
│   └── figures/       PNG + SVG
├── 04_manuscript/
│   ├── manuscript.md  원고 단일 원본
│   ├── references.bib
│   └── submission/    투고 세트
├── 05_talk/
│   ├── outline.md     제목 = 주장
│   ├── deck/          HTML · PDF · PPTX
│   ├── script.md      대본
│   └── cuecards/      인쇄용 PDF
└── notes/
    ├── decision.md    왜 이렇게 했는지
    └── todo.md
```

왜 이렇게 나누나

raw는 성역입니다. 원본 데이터를 고치면 재현이 불가능해집니다. AI에게도 “raw는 읽기만”이라고 명시하세요. OpenCode는 파일 권한을 물어보지만, 사람이 규칙을 먼저 정하는 편이 안전합니다.

derived는 코드가 만듭니다. 손으로 만든 엑셀은 6개월 뒤에 설명할 수 없습니다. 정제 단계부터 코드로 남기면, 심사자가 “이 변수는 어떻게 정의했나요?”라고 물을 때 파일 하나를 보여주면 됩니다.

manuscript는 한 파일입니다. `manuscript_v3_final_final.docx`가 다섯 개가 되는 순간 끝입니다. 마크다운 한 파일을 원본으로 두고, 제출용 docx는 매번 새로 굽습니다.

notes/decision.md가 가장 값어치 있습니다. “가중치는 연도별로 나눴다”, “이 변수는 2019년부터 문항이 바뀌어 통합하지 않았다” 같은 판단을 적어 두면, 3개월 뒤의 나와 심사자가 같은 문서를 봅니다.

첫 폴더를 만드는 프롬프트

빈 폴더를 열고 아래를 보내면 위 구조를 만들어 줍니다. 만든 뒤에는 각 폴더에 무엇을 넣을지를 `README.md`에 적게 하세요.

PROMPT

```
지금 폴더에 논문 한 편을 위한 작업 구조를 만들어줘.
위 구조를 참고하되, 파일은 만들지 말고 폴더와 README.md만.
README.md에는 각 폴더의 역할과 "raw는 수정 금지" 규칙을 적어줘.
만들기 전에 어떤 폴더를 만들지 먼저 보여줘.
```

파일 이름 규칙 — 나중의 나를 위한 배려

같은 폴더를 6개월 뒤에 열어도 이해되는 이름

대상	규칙	좋은 예	피할 예
원고	역할-버전	manuscript_v3.md	최종최종_진짜.docx
표	번호-내용	table1_baseline.docx	표1_수정.docx
그림	번호-내용-형식	fig2_forest.svg	그림2.png
분석 코드	순서-목적	03_models.R	analysis_final.R
발표	매체-버전	deck_v4.html	발표_최종.pptx

날짜를 붙이는 습관도 좋습니다. `2026-10-25_decision.md`처럼 앞에 두면 정렬이 됩니다.

04 / SKILLS

일곱 개의 스킬,
일곱 가지 역할

스킬은 “프롬프트 모음”이 아니라 **일하는 절차서**입니다.

스킬은 AI에게 주는 업무 매뉴얼입니다. 그냥 “논문 써줘”라고 하면 평범한 답이 오지만, `@academic-research-writing`을 붙이면 근거 장부를 만들고, 인용을 검증하고, 숫자를 표로 보내는 절차를 따릅니다.

`@academic-research-writing`

논문·문헌고찰

연구 질문을 계약서로 만들고, 근거 장부(ledger)를 쌓고, IMRaD 원고를 씁니다. 인용은 “존재 확인”과 “주장 지지 확인”을 따로 합니다.

쓸 때: 원고 초안, 문헌고찰, 체계적 고찰, 투고용 리포트

`@deep-research`

조사·팩트체크

질문을 쪼개 넓게 찾고 좁히며, 모든 주장에 출처를 붙입니다. “최신 동향” “비교해줘” “사실 확인”의 기본 경로입니다.

쓸 때: 배경 조사, 가이드라인 비교, 통계 수치 확인

`@paper-review`

제출 전 자체 심사

이미 쓴 원고·표·그림·발표자료를 심사자 관점에서 감사합니다. AI 티, 숫자 나열, 인용·표·그림 번호, 덱 커버리지를 봅니다.

쓸 때: 투고 1주 전, 발표 3일 전

`@design-craft`

보이는 것의 설계

보고서·포스터·슬라이드의 색·타이포·여백·밀도를 정하고, 검사 스크립트로 넘침과 대비를 확인합니다.
표면마다 테마를 새로 뽑습니다.

쓸 때: PDF·HTML·포스터를 사람에게 건넬 때

`@deck-craft`

발표자료

16:9 고정, 한 장에 한 아이디어, 제목은 주장. 도표는 이미지 생성이 아니라 SVG로 그립니다. HTML·PPTX·PDF를 함께 만듭니다.

쓸 때: 학회·교원 연수·IR 발표

`@script-to-cuecards`

대본 → 큐카드

대본을 A4 두 장 분할 카드로 인쇄합니다. 넘치면 글자를 줄이지 않고 "계속" 카드로 이어 붙입니다.

쓸 때: 리허설, 사회자·발표자 지참 카드

`@clinical-prescribing-safety`

임상 근거·안전

처방·용량·금기·상호작용 질문에 근거 등급과 출처를 붙입니다. 최종 처방은 사람이 서명합니다.

쓸 때: 임상 사례, 약물 검토, 진료 지원

부르는 방법은 두 가지

①
직접 부르기

: `@academic-research-writing` 처럼 아이디를 씁니다. ②
맡기기

: "논문 원고를 심사자 관점에서 봐줘"라고 하면 AI가 알맞은 스킬을 스스로 고릅니다. 처음에는 ①이 예측 가능해서 좋습니다.

스킬을 부르는 순서 — 겹칠 때의 규칙

상황	먼저	다음	이유
원고를 쓰고 표를 만든다	academic-research-writing	design-craft	내용이 먼저, 표면은 나중
발표자료를 만든다	deck-craft	script-to-cuecards	슬라이드가 정해져야 대본이 맞음
약물·용량이 나온다	clinical-prescribing-safety	academic-research-writing	환자 안전이 문체보다 우선
“이거 왜 이렇지?” 조사	deep-research	academic-research-writing	조사와 문서 작성은 다른 일

05 / WORKFLOW

여덟 단계, 질문에서 큐카드까지

각 단계는 “내가 하는 일”과 “AI가 하는 일”로 나뉩니다.

아래 순서를 지키면 중간에 방향을 잃지 않습니다. 특히 **3단계(분석 계획)를 건너뛰지 마세요.** 코드부터 짜면 표가 나온 뒤에 질문을 바꾸게 됩니다.

STEP 1 질문을 계약서로	STEP 2 자료·문헌을 한곳에
STEP 3 분석 계획 고정	STEP 4 표 1 · 주 분석
STEP 5	STEP 6

원고 IMRaD	자체 심사
STEP 7 투고 패키지	STEP 8 발표 · 큐카드

1 질문을 계약서로 바꿉니다

논문의 성패는 첫 문장에서 갈립니다. "~에 대해 알아보고자 했다"는 질문이 아닙니다.

내가 하는 일

- PICO(대상·중재/노출·비교·결과)를 한 문장으로 씁니다.
- 이 연구가 **기술(descriptive)**인지 **연관(association)**인지 **예측(prediction)**인지 정합니다.
- "이 결과가 나오면 무엇을 다르게 할 것인가"를 한 줄 적습니다.

프롬프트

PROMPT

```
@academic-research-writing
아래 연구 아이디어를 PICO로 정리하고, 연구 설계 후보 2개를
장단점과 함께 제시해줘. 아직 원고는 쓰지 마.

아이디어: 국민건강영양조사 자료에서 40세 이상 성인의
복부비만과 대사증후군 유병률의 연도별 추이를 보고 싶다.
목표 저널: 가정의학과 국내 학술지.
```

AI가 하는 일

PICO 표, 설계 후보(단면/추세), 필요한 변수 목록, 윤리·보고 지침(STROBE) 체크리스트를 만듭니다. 이 답을 그대로 쓰지 말고 **한 문장씩 소리 내어 읽어 보세요.** 말이 안 되면 질문이 아직 흐릿한 것입니다.

끝났다는 신호

연구 질문·설계·주 결과·보고 지침이 `notes/decision.md`에 10줄 이내로 적혀 있으면 다음 단계로 갑니다.

2 자료와 문헌을 한 폴더에 모읍니다

AI가 찾아야 할 것을 찾을 수 있게, 책상을 먼저 정리합니다.

내가 하는 일

- 원시자료·코드북·설문지를 `01_data/` 에 넣습니다. **원본은 읽기 전용**으로 표시합니다.
- 핵심 문헌 5~10편의 PDF를 `04_manuscript/refs/` 에 넣습니다.
- 목표 저널의 **저자 지침을** PDF로 저장합니다(분량·표 개수·인용 형식).

프롬프트

PROMPT

```
@academic-research-writing
01_data/codebook 폴더의 코드북을 읽고,
내 연구 질문에 필요한 변수 후보를 "변수명 / 정의 / 척도 / 결측 가능성" 표로 정리해줘.
코드북에 없는 변수는 추측하지 말고 "코드북에 없음"이라고 적어.
그다음 04_manuscript/refs의 PDF 5편을 읽고, 각 논문이 내 질문에
어떤 근거를 주는지 한 문단씩 요약해줘. 인용은 파일명과 쪽수를 남겨.
```

> **PDF를 읽었다고 말할 때 확인할 것**
> AI가 “전문을 읽었다”고 해도 실제로는 초록만 봤을 수 있습니다. 논문마다 `access_level: full_text | abstract | metadata_only` 를 적게 하세요. 이 습관이 나중에 심사위원의 질문을 막아 줍니다.

---

3 분석 계획을 코드보다 먼저 고정합니다

“돌려 보니 유의했다”는 연구가 아니라, “이렇게 분석하겠다고 정했고, 그대로 했다”가 연구입니다.

내가 하는 일

- 주 결과(primary outcome)와 보조 결과를 구분합니다.
- 포함·제외 기준, 결측 처리, 보정 변수를 정합니다.
- 민감도 분석을 1~2개 미리 정합니다.

프롬프트

PROMPT

```
@academic-research-writing
아래 분석 계획 초안을 STROBE 체크리스트에 비추어 검토해줘.
빠진 항목, 애매한 표현, 나중에 심사에서 문제될 부분을
"위험도 높음/중간/낮음"으로 표시하고, 각각 수정 문장을 제안해줘.
코드는 아직 쓰지 마.

초안:
- 대상: 2019-2021년 국민건강영양조사 40세 이상
- 노출: 복부비만(허리둘레 기준)
- 결과: 대사증후군
- 보정: 나이, 성별, 흡연, 음주, 교육, 소득
- 분석: 복합표본 로지스틱 회귀
```

이 단계의 산출물은 `02_analysis/plan.md` 입니다. 나중에 분석을 바꾸면 **왜 바꿨는지를** 이 파일에 날짜와 함께 적습니다. 그것이 "사후 분석"과 "분석 계획 위반"을 구분해 줍니다.

---

4 표 1과 주 분석을 만듭니다

표가 먼저입니다. 문장은 표를 설명하는 역할만 합니다.

내가 하는 일

- "표 1에 들어갈 변수"를 먼저 정합니다(보통 대상 특성 + 노출 + 결과).
- AI가 만든 숫자를 **원자료에서 한 번 더** 확인합니다.
- 코드·로그·출력 표를 같은 폴더에 남깁니다.

프롬프트

PROMPT

```
@academic-research-writing
02_analysis/plan.md의 계획대로 R 코드를 작성해줘.
조건:
1. 원자료는 읽기만 하고 절대 수정하지 마.
2. 국민건강영양조사의 복합표본설계(층 · 집락 · 가중치)를 반드시 반영해.
   변수명은 코드북에서 확인하고, 확인한 이름을 주석에 적어.
3. 표 1은 연속변수는 평균(표준오차) 또는 중앙값(IQR),
   범주형은 가중 백분율로 만들어줘.
4. 코드 실행 후 logs/에 세션 정보와 패키지 버전을 남겨.
5. 결과를 03_output/tables/table1.csv 로 저장해.
먼저 코드를 보여주고, 내가 승인하면 실행해.
```

> **절대 하지 말 것**
> AI가 만든 숫자를 원자료 확인 없이 원고에 옮기는 것. 특히 **가중치를 빼먹은 백분율은** KNHANES에서 가장 흔한 치명적 오류입니다. 표 1의 n과 원자료 n이 맞는지, 가중 %의 합이 100인지 확인하세요.

5 원고를 IMRaD로 씁니다

서론은 "왜 이 질문인가", 방법은 "재현 가능하게", 결과는 "표를 가리키며", 고찰은 "한계까지".

내가 하는 일

- 결과 섹션에는 숫자를 최소로 쓰고 표를 가리킵니다.
- 고찰에는 **한계를** 먼저 씁니다. 심사자가 찾기 전에 내가 찾습니다.
- AI가 쓴 문장은 반드시 소리 내어 읽고, 내 말투로 고칩니다.

프롬프트

PROMPT

```
@academic-research-writing
03_output/tables/table1.csv와 02_analysis/plan.md를 근거로
원고의 Results 섹션을 써줘. 규칙:
- 문장 하나에 숫자는 최대 하나. 나머지는 표를 가리켜.
- 표에 없는 숫자는 절대 만들지 마.
- "유의했다" 대신 추정치와 95% CI, p값을 표에 두고
  문장에서는 방향과 크기만 말해.
- 한국어로 쓰고, 마지막에 영어 초록용 3문장을 따로 붙여.
쓰기 전에 어떤 표를 근거로 쓸지 먼저 확인해.
```

원고 파일은 `04_manuscript/manuscript.md` 하나만 씁니다. 버전은 파일명이 아니라 git이나 날짜 폴더로 관리합니다.

6 제출 전에 심사자를 먼저 부릅니다

투고 1주 전, 스스로에게 리뷰를 받습니다. 이 단계에서 걸러지지 않은 것은 심사자에게 걸립니다.

프롬프트

PROMPT

```
@paper-review
04_manuscript/manuscript.md, 03_output/tables/, 03_output/figures/,
05_talk/deck/ 를 함께 읽고 심사자 관점에서 감사해줘.
순서:
1) 주장과 근거의 대응(claim ↔ evidence) - 근거 없는 단정
2) 숫자 나열·과장·애매한 표현
3) 표·그림·참고문헌 번호와 본문 삽입 위치
4) STROBE 항목 누락
5) 발표자료가 논문의 핵심을 실제로 담고 있는지
결과는 HIGH/MED/LOW로 나누고, 각 항목에 파일·줄·수정안 3줄을 붙여.
```

이 스킬은 "고쳐 주는" 스킬이 아닙니다

먼저
지적

하게 하고, 내가 동의한 항목만 고치게 하세요. 전부 자동 수정하면 내가 이해하지 못한 문장이 원고에 남습니다.

7 투고 패키지를 한 폴더에 고정합니다

저널이 요구하는 파일을 이름 그대로 만듭니다. 제출 화면에서 헤매지 않습니다.

PROMPT

```
@academic-research-writing
목표 저널의 저자 지침(04_manuscript/refs/journal_guide.pdf)을 읽고,
투고에 필요한 파일 체크리스트를 만들어줘.
그다음 04_manuscript/submission/ 폴더에 아래를 준비해:
- 표지(제목·저자·소속·교신저자)
- 본문(맹검판: 저자 정보 제거)
- 표·그림(저널 요구 형식·해상도)
- Cover letter
- 이해상충·AI 사용 공개 문구
각 파일이 준비됐는지 표로 보고하고, 빠진 것은 "없음"이라고 표시해.
```

- **맹검판**과 **비맹검판을** 따로 만듭니다.
- 표·그림은 본문과 별도 파일로, 캡션은 한 파일에 모읍니다.

- AI 사용 공개는 저널 규정을 먼저 확인합니다(아래 9장).

---

8 발표자료 → 대본 → 큐카드 → 리허설

발표는 "슬라이드를 예쁘게"가 아니라 "8분 동안 논리 하나를 완주"하는 일입니다.

① 아웃라인 — 제목만

PROMPT

```
@deck-craft
내 논문(04_manuscript/manuscript.md)을 8분 학회 발표로 만들 거야.
먼저 슬라이드 제목만 13개 나열해줘. 각 제목은 "주장" 문장이어야 해.
슬라이드 본문·디자인은 아직 만들지 마.
청중: 가정의학과 · 내과, 절반은 비전문가.
```

제목 목록을 보고 **"이 순서로 말하면 논리가 서는가"를** 확인합니다. 마음에 안 들면 여기서 고칩니다. 슬라이드를 만든 뒤 고치는 것보다 10배 빠릅니다.

② 덱 — 한 장에 한 아이디어

PROMPT

```
@deck-craft
승인한 아웃라인으로 16:9 HTML 발표자료를 만들어줘.
- 1920×1080, 오프라인에서 열려야 함
- 숫자는 표·도표에, 제목에는 최대 하나
- 도표는 SVG로 직접 그려줘(이미지 생성 금지)
- 마지막에 참고문헌 · 이해상충 슬라이드
- 발표자 노트도 슬라이드마다 30-45초 분량으로
```

③ 대본 — 말하는 시간

PROMPT

```
@deck-craft
슬라이드별로 말할 대본을 써줘. 총 8분, 전환 25초 포함.
각 슬라이드에 "시간 / 말할 것 / 넘어가는 한 문장"을 붙여.
숫자는 꼭 필요한 것만 소리 내어 읽고, 나머지는 "표를 보시면"으로 넘겨.
어려운 용어는 첫 등장에 한 줄로 풀어줘.
```

④ 큐카드 — 손에 들고 보는 것

PROMPT

```
@script-to-cuecards
05_talk/script.md로 A4 세로 2분할 큐카드 PDF를 만들어줘.
발표 파트와 질의응답 파트를 따로 만들어.
인쇄는 단면 · 실제 크기 100%로 할 거야.
```

⑤ 리허설 — 타이머

- 1회차: 시간 재지 말고 끊김 없이 한 번.
- 2회차: 타이머. 목표 ±10%면 통과.
- 3회차: 예상 질문 5개에 소리 내어 답하기.
- 발표장에서는 **PDF를** 띄웁니다. 폰트·애니메이션이 발표 PC에서 달라질 수 있습니다.

8분 = 13장인 이유

슬라이드당 30~45초가 편안한 속도입니다. 8분이면 도입·마무리 여유를 빼고 11~14장. 20분이면 28~35장이 아니라

24장 안팎이

적당합니다. 장수를 늘리는 것보다 한 장을 오래 보는 편이 낫습니다.

---

06 / KNHANES

국민건강영양조사는 설계가 절반입니다

AI가 가장 자주 틀리는 지점이 바로 여기입니다.

국민건강영양조사는 "전국 대표성"을 위해 **층화·집락·가중치**를 쓰는 복합표본조사입니다. 이 설계를 빼고 `mean()`을 돌리면 숫자는 나오지만, 그 숫자는 **한국 국민의 값이 아닙니다.**

왜 가중치를 써야 하나

조사에 참여한 사람이 국민 전체를 대표하도록 각 참여자에게 **가중치를** 줍니다. 가중치를 빼면 젊은 층이 과대표되고, 유병률이 실제와 달라집니다. 층·집락을 빼면 표준오차가 과소평가되어 **p값이 실제보다 작아집니다.** 즉, 없는 유의성이 생깁니다.

KNHANES 원시자료에는 보통 층(`kstrata`), 집락(`psu`), 가중치(`wt_*`)가 들어 있습니다. 정확한 변수명은 **연도·파일마다 다르므로** 반드시 코드북에서 확인해야 합니다.

> **AI에게 반드시 말할 한 문장**
> "이 분석은 복합표본설계입니다. 층·집락·가중치를 반영하고, 변수명은 코드북에서 확인한 뒤 주석에 적으세요. 가중치를 빼면 결과를 폐기하겠습니다."
>
> 이 문장을 매 분석 프롬프트에 붙이세요. 한 번 말하고 잊으면, 세 번째 스크립트에서 조용히 빠집니다.

코드의 모양 — R과 Stata

아래는 **구조를 보여 주는 예시**입니다. 변수명은 여러분의 연도·파일 코드북에서 확인한 이름으로 바꿔야 합니다. 그대로 복사해 실행하면 안 됩니다.

R · SURVEY 패키지

PROMPT

```
# 변수명은 코드북에서 확인 - 아래는 자리표시자
library(survey)
library(dplyr)

raw <- readRDS("01_data/raw/knhanes_2019.rds")

des <- svydesign(
  ids     = ~psu,        # 집락
  strata  = ~kstrata,    # 층
  weights = ~wt_exam,    # 검진 가중치
  nest    = TRUE,
  data    = raw
)

# 가중 유병률 + 95% CI
svymean(~factor(htn), des, na.rm = TRUE)

# 복합표본 로지스틱 회귀
fit <- svyglm(htn ~ age + sex + bmi + smoking,
              design = des, family = quasibinomial())
summary(fit)
confint(fit)
```

STATA

PROMPT

```
* 변수명은 코드북에서 확인
use "01_data/raw/knhanes_2019.dta", clear

svyset psu [pweight=wt_exam], strata(kstrata)

svy: mean htn
svy: proportion htn
svy: logistic htn age sex bmi smoking

* 연도별로 나누어 분석할 때
svyset psu [pweight=wt_exam], strata(kstrata)
svy, subpop(if year==2019): mean htn
```

연도를 합칠 때 — 가장 위험한 부분
서로 다른 연도의 원시자료를 단순히 `rbind` 하고 한 번에 분석하면 가중치가 틀어집니다. 연도별 가중치를 합치는 방식, 연도를 층에 넣는 방식, 통합가중치를 쓰는 방식이 각각 다르고, **어떤 방식을 택했는지 방법에 써야** 합니다. 확신이 없으면 **통계 담당자에게 확인**하고, 그 결정을 `notes/decision.md` 에 남기세요. AI에게 "알아서 해줘"라고 하면 안 됩니다.

흔한 실수 일곱 가지

#	실수	결과	예방 프롬프트
1	가중치를 빼고 `mean()`	유병률이 실제와 다름	"복합표본설계를 반영했는지 코드로 증명해"
2	층·집락 무시	p값 과소, 거짓 유의	"svydesign/svyset 부분을 보여줘"
3	연도 통합 가중치 임의 처리	대표성 상실	"통합 방식과 근거를 문서로 남겨"
4	결측을 0으로 채움	추정치 왜곡	"결측 수와 처리 방법을 표로 보고해"
5	설문·검진 가중치 혼용	분석마다 n이 달라짐	"이 분석에 맞는 가중치 변수를 코드북에서 확인해"
6	범주를 합쳐 결과를 만든 뒤 "사전 계획"이라 함	사후 분석 은폐	"분석 전 계획과 실제 코드의 차이를 표로"
7	코드북 버전 불일치	변수 정의 오류	"코드북 파일명·연도를 주석에 적어"

연습: 숫자 없는 리허설

처음에는 실제 분석을 돌리지 말고,
"이 계획대로라면 어떤 표가 나와야 하는가"를

빈 표로 그려 보세요. 행과 열이 정해지면 AI가 채울 곳과 내가 확인할 곳이 분명해집니다. 빈 표를 먼저 만드는 습관이 가중치 실수를 절반 줄입니다.

07 / CASE

HCC vs 혈관종: 무엇을, 어떻게 했나

2026-07 ~ 2026-10, 한 저장소에 남은 실제 기록.

제 연구는 국민건강영양조사가 아니라 **공개 초음파 데이터셋**을 썼습니다. 하지만 일하는 방식은 여러분의 표 데이터 연구와 같습니다. 무엇이 달랐는지, 어디서 막혔는지가 여러분께 더 유용합니다.

한 장 요약

질문	B-mode 초음파에서 HCC와 혈관종을 구분할 때, 단일 확률값 외에 **"얼마나 HCC다운가"**를 연속값으로 줄 수 있는가
데이터	SMC-LUD 공개 데이터셋 · clean subset 2,656장 / 744명 · 환자 단위 분할
모델	CNN–Transformer 하이브리드(EfficientNetV2B0 + transformer) · SupCon 학습
주 결과	테스트 268장 · 민감도 95.71% · 특이도 100% · 정확도 97.76%
정직한 한계	닮음 지표의 불일치 6 vs 1, McNemar exact **p = 0.125** → "우월하다"고 말하지 않음
발표	13장 · 8분 + 질의 2분 · 대본 7분 35초 · 큐카드 19장

시간선

2026-07

데이터와 모델

공개 데이터셋 정제, 환자 단위 분할, 기준 모델 학습. 이 단계가 전체의 절반 이상.

2026-08

주 결과와 표

혼동행렬, ROC, 컷오프. “0.50이 왜 0.50인가”를 설명하는 데 가장 오래 걸림.

2026-09

원고와 자체 심사

IMRaD 초안 → 논문 투고 → 리뷰 대응. 수치 감사에서 여러 번 되돌아감.

2026-10

발표 패키지

아웃라인 → 13장 덱 → 대본 v5 → 큐카드 → 리허설. 발표 후 수상.

가장 오래 걸린 일은 “숫자를 포기하는 것”이었습니다

처음에는 “닮음 점수가 확률 점수보다 낫다”는 이야기를 하고 싶었습니다. 그런데 테스트 결과는 **6대 1**로 닮음 점수가 더 많은 오류를 냈고, McNemar 검정은 **p = 0.125**였습니다. 유의하지 않았습니다.

AI에게 “이걸 어떻게 하면 유의하게 만들 수 있나”라고 묻고 싶었습니다. 대신 이렇게 물었습니다. **“이 결과로 말할 수 있는 것과 말할 수 없는 것을 표로 나눠 줘.”** 그 표가 발표의 11번째 슬라이드가 되었고, 오히려 심사위원의 신뢰를 얻었습니다.

> “차이는 유의하지 않습니다. 다만 실패하는 지점이 다릅니다.”
>
> 발표 11장의 제목. 이 한 문장이 발표의 성격을 바꿨습니다.

여러분의 연구에 옮길 수 있는 세 가지

① 컷오프는 점이 아니라
구간

으로 설명한다(검증에서 오류가 0이 되는 구간). ② 새 지표가 기존 지표를 이기지 못하면
“어디서 다른 방식으로 실패하는가”

로 논점을 바꾼다. ③ 발표에는
한계 슬라이드를

반드시 넣고, 질의에서 그 슬라이드로 돌아간다.

07.5 / AWARD

상을 목표로 한다면,
심사위원의 질문을 먼저 설계합니다

좋은 연구를 설명하는 것과 상을 받는 것은 다른 기술입니다.

제가 받은 질문은 통계가 아니었습니다. **IRB를 통과했는가,**
이 연구를 계속할 것인가, 임상에서 어디에 쓸 수 있는가. 세
질문 모두 “비판”이 아니라 **다음 단계를 묻는 질문이**었습니다.

질문 1

“IRB는 통과했나요?”

심사위원은 연구의 윤리적 근거를 확인합니다. 답이 흐리면 나머지 점수도 흔들립니다.

질문 2

“이거 계속 연구할 건가요?”

일회성 발표인지, 연구 프로그램인지 봅니다. 다음 질문이 있는 사람이 높게 평가받습니다.

질문 3

“임상에서 어디에 쓰나요?”

성능표가 아니라 **진료 흐름의 어느 지점을** 바꾸는지 묻습니다.

세 질문에 대한 답변 틀

1 IRB — “통과했습니다”가 아니라 “이 근거로 면제/승인받았습니다”

준비물: 승인번호, 승인일, 기관명, 연구 유형(전향/후향), 동의 면제 사유. 공개 데이터셋이면 **심의면제 확인서를** 지참합니다.

30초 답변 예시

“본 연구는 OO병원 IRB에서 승인번호 OOOO로 승인받았습니다. 후향적 검사 데이터를 익명화해 사용했고, 환자 동의는 면제되었습니다. 공개 데이터셋 부분은 국가 공개 자료로 기관 심의면제 확인을 받았습니다.”

> **말하지 말아야 할 것**
> “IRB가 필요 없는 연구였어요.” 필요 없다고 단정하기보다 **어떤 근거로 면제받았는지를** 말합니다. 기관마다 요구가 다르므로 발표 전에 확인서를 출력해 가방에 넣어 두세요.

2 계속 연구 — “네”보다 “이렇게 하겠습니다”

구체적인 1년 계획을 2~3문장으로 준비합니다. 하고 싶은 것만 말하면 산만해 보이고, **하지 않을 것을** 함께 말하면 신뢰가 생깁니다.

30초 답변 예시

“다음 단계는 외부 병원 코호트로 검증하는 것입니다. 올해는 전향적으로 200예를 모을 계획이고, 다기관으로 확장하기 전에 이 기관에서 재현성을 먼저 확인하겠습니다. 상용화나 규제 승인은 아직 계획에 없습니다.”

3 임상 응용 — “정확도”가 아니라 “워크플로의 어느 칸”

“판독을 대체한다”가 아니라 “판독 전에 위험을 표시한다”, “검진에서 재검 대상자를 좁힌다”, “교육에서 전형과 비전형을 보여 준다”처럼 **한 칸을** 지정합니다.

30초 답변 예시

“저는 이 지표를 판독 대체가 아니라 보조로 봅니다. 초음파 판독 전에 ‘전형 HCC에서 얼마나 먼가’를 보여 주면, 비전형 증례를 놓치지 않게 도울 수 있습니다. 다만 전향적 검증 전에는 임상 결정에 쓰지 말아야 한다고 생각합니다.”

어떤 데이터로 하는 연구가 상을 받나

올해 수상 사례를 보면 **국민건강영양조사 2차 분석만으로는 상을 받기 어렵습니다.** 실제로 상을 받은 연구들은 대체로 두 갈래였습니다.

갈래 A · 원내 코호트

내 기관에서 직접 쌓은 데이터

예: 검진센터에서 구축한 *H. pylori* 위염 후향 코호트(올해 다른 수상 사례). 데이터를 **만든 것 자체가** 기여입니다. 검진센터·외래·내시경실 데이터를 연구 가능한 형태로 정리하는 일이 핵심입니다.

갈래 B · 공개 데이터셋

재현 가능한 벤치마크

예: 제 연구(SMC-LUD). 누구나 같은 데이터로 재현할 수 있어 **방법론의 기여가** 분명합니다. 대신 "왜 이 데이터여야 하는가"와 "외부 타당도"를 반드시 답해야 합니다.

데이터 전략별 장단점 — 발표 기획 단계에서 정합니다

데이터	강점	약점	수상 전략
국민건강영양조사	전국 대표성, 무료, 표본 큼	2차 분석이라 독창성 약함, 인과 추론 제한, 변수 제한	정책·인구집단 질문 + 연도 추세 + 다른 데이터와 연결. "처음 규명한 인구집단"을 전면에.
원내 후향 코호트	우리 기관만의 데이터, 임상 맥락 강함	선택 편향, 결측, IRB·개인정보 부담	코호트 구축 과정과 정의를 방법의 기여로. 임상 워크플로 변화를 보여 줌.
공개 데이터셋	재현 가능, 벤치마크, 빠른 시작	독창성 경쟁, 외부 타당도 의문	방법론 기여 + 외부 검증 계획 + "이 데이터의 한계"를 먼저 말함.

국민건강영양조사로 상을 노린다면

① "유병률 조사"에서 멈추지 말고 정책이 바뀌어야 하는 이유까지 간다. ② 한 해가 아니라 여러 해의 추세를 본다. ③ 가능하면 원내 자료나 다른 공개 자료와 연결해 "검증"을 붙인다. ④ 발표 제목에 대상·기간·핵심 발견을 넣는다. "한국 성인의 OO 추이: 2010–2023"처럼.

수상을 노리는 발표의 구조 — 8분 기준

시간	슬라이드	반드시 들어갈 것
0:00-0:40	표지 + 문제	“왜 지금 이 질문인가” 한 문장. 숫자 하나.
0:40-1:40	배경 + 질문	기존 방법의 한계, 이 연구가 메우는 빈칸.
1:40-3:10	방법	데이터·설계·IRB를 한 장에. 심사위원이 여기서 질문을 만든다.
3:10-5:40	결과	표 1~2개, 그림 1개. 숫자는 표에.
5:40-6:40	한계	내가 먼저 말한다. “이건 말할 수 없습니다.”
6:40-7:40	다음 연구 + 임상 응용	1년 계획 2문장, 워크플로 한 칸.
7:40-8:00	감사	이해상충·AI 사용 공개.

> “심사위원은 틀린 곳을 찾으러 오지 않습니다. 이 사람과 다음 이야기를 할 수 있는지 보러 옵니다.”
>
> 발표를 준비하며 바꾼 마음가짐

08 / CONVERSATION

대화 규칙 열둘 — hand-to-hand로 일하는 법

프롬프트 기술보다 **일을 나누는 순서가** 중요합니다.

AI와 일할 때 실패하는 이유는 대개 모델 성능이 아니라 **한 번에 너무 많이 시키는 것**입니다. 아래 열두 규칙은 제가 네 달 동안 지킨 것이고, 어기면 반드시 되돌아왔습니다.

1. 폴더를 열고 시작한다

“내 컴퓨터에 있는 논문”이라고 말하지 않습니다. 폴더를 열어 **현재 위치를** 맞춥니다. AI가 엉뚱한 파일을 읽는 사고의 90%가 여기서 납니다.

2. 계획을 먼저 받는다

“고쳐줘” 전에 “어떻게 고칠지 먼저 보여줘”라고 합니다. 계획이 틀렸으면 파일을 건드리기 전에 멈출 수 있습니다.

PROMPT

```
먼저 계획만 보여줘. 파일은 아직 고치지 마.
계획을 읽고 내가 승인하면 그때 실행해.
```

3. 한 번에 하나

“표 1, 표 2, 그림 3, 원고를 한 번에”는 실패합니다. 한 항목을 끝내고 **결과를 확인한 뒤** 다음으로 갑니다. 대화가 길어지는 것보다 짧은 왕복이 빠릅니다.

4. 결정은 파일로 남긴다

채팅은 사라집니다. “가중치는 연도별로 나눴다” 같은 결정은 `notes/decision.md`에 날짜와 함께 적게 합니다. 3개월 뒤의 내가 읽을 수 있어야 합니다.

5. 숫자는 표로 보낸다

문장에 숫자를 넣으면 AI가 표와 다른 숫자를 써도 눈치채기 어렵습니다. **문장 하나에 숫자 하나**, 나머지는 “표 2를 보시면”으로 넘깁니다.

6. 출처 없는 문장은 쓰지 않는다

서론·고찰의 모든 근거 문장에는 인용이 붙어야 합니다. AI가 "잘 알려져 있다"고 하면 **"어디에?"**라고 묻습니다. 답이 없으면 그 문장은 삭제합니다.

7. "모른다"고 말하게 한다

프롬프트에 이 문장을 넣으세요: **"확실하지 않으면 '확인 필요'라고 적고 추측하지 마."** AI는 빈칸을 채우려는 습성이 있습니다. 빈칸을 남기는 것이 실력입니다.

8. 되돌릴 수 있게 만든다

큰 수정 전에는 폴더를 복사하거나 git 커밋을 합니다. "이전 버전으로 돌려줘"는 **파일이 남아 있을 때만** 가능합니다.

PROMPT

```
지금 상태를 git에 커밋해줘. 메시지는 "before table revision".
그다음 수정을 시작해.
```

9. 내 말투로 고친다

AI 문장은 매끄럽지만 나와 닮지 않았습니다. 소리 내어 읽고 **내가 실제로 말할 문장**으로 바꿉니다. 발표 대본은 특히 그렇습니다.

10. 검증 질문을 던진다

"이 숫자의 출처 파일이 뭐야?", "이 표의 n은 어디서 왔어?", "이 인용이 그 주장을 실제로 지지해?" 답을 파일로 보여 주지 못하면 믿지 않습니다.

11. 길어지면 새로 시작한다

대화가 2시간을 넘으면 앞 내용을 잊습니다. **“지금까지 결정·경로·다음 할 일을 handoff.md로 정리해줘”**라고 한 뒤 새 세션을 시작합니다.

12. AI는 저자가 아니다

AI 사용은 방법이나 감사의 말에 공개하고, 저자 자격은 없습니다. 최종 책임은 사람에게 있습니다. 저널마다 규정이 다르니 투고 전에 확인합니다.

나쁜 프롬프트를 좋은 프롬프트로

나쁜 예	왜 실패하나	좋은 예
“논문 써줘”	질문·데이터·저널이 없음	“이 표와 계획을 근거로 Results만, 숫자는 표로 넘기고”
“통계 해줘”	설계·가중치·결과 정의가 없음	“복합표본설계로, 이 outcome, 이 보정변수, 이 결측 처리로”
“예쁘게 만들어줘”	취향 판정 불가	“학회 청중, 16:9, 세리프 제목, 악센트 한 색, 표는 24pt 이상”
“다 고쳐줘”	내가 이해 못한 문장이 남음	“문제를 HIGH/MED/LOW로 나열하고, 내가 승인한 것만 고쳐”
“이게 맞아?”	무엇이 맞는지 불명확	“이 숫자가 원자료와 일치하는지, 출처 파일과 함께 보여줘”

09 / SAFETY

믿지 않을 것은 믿지 않기

AI는 빈칸을 채우고, 사람은 빈칸을 남깁니다.

AI가 만든 원고에서 가장 위험한 것은 문장이 아니라 **숫자와 인용**입니다. 문장은 읽으면 이상한 티가 나지만, 숫자와 DOI는 그럴듯해서 그냥 지나갑니다.

환각 — 세 가지 얼굴

- **없는 논문** — 저자·연도·DOI가 모두 그럴듯하지만 존재하지 않습니다.
- **맞는 논문, 틀린 주장** — 논문은 있지만 내 주장을 지지하지 않습니다.
- **그럴듯한 숫자** — 표에 없는 값이 본문에 등장합니다.

대응은 하나입니다. **원문·원자료로 돌아가기.** "DOI 확인"은 "주장 지지 확인"이 아닙니다.

인용 검증 3종 세트

PROMPT

```
이 원고의 참고문헌을 하나씩 검증해줘.
각 항목에 대해:
1. 논문이 실제로 존재하는가 (DOI/PMID로 확인)
2. 내가 인용한 문장이 그 논문의 어느 부분에서 지지되는가 (쪽 · 표)
3. 접근 수준 (full_text / abstract / metadata_only)
셋 중 하나라도 확인 못 하면 "확인 불가"로 표시하고,
그 인용을 쓰지 말라고 표시해.
```

개인정보 — 절대 규칙

- 환자 이름·등록번호·생년월일·연락처·주소는 **어떤 채팅에도** 넣지 않습니다.
- 원내 데이터는 **기관 정책을** 먼저 확인합니다. 외부 모델 제공자로 전송될 수 있습니다.
- 국민건강영양조사 원시자료는 공개 자료지만, **재배포·업로드는** 이용 약관을 확인합니다.
- 스크린샷에는 종종 환자 정보가 함께 찍힙니다. 올리기 전에 잘라냅니다.

AI 사용 공개

저널·학회마다 문구가 다릅니다. 보통 “English language editing”인지 “analysis support”인지 구분해 적습니다. **AI는 저자가 될 수 없고**, 분석을 했다면 방법에 적습니다. 제출 전에 목표 저널의 최신 지침을 확인하세요.

통계 — AI에게 맡기면 안 되는 것

- 표본 크기·검정력 계산의 가정
- 복합표본 가중치·층·집락 처리
- 다중비교 보정, 결측 처리, 민감도 분석
- “유의하다”의 해석

AI는 코드를 빨리 쓰지만, **설계의 타당성은** 검증하지 못합니다. 통계 담당자와 한 번 더 확인하세요.

제가 실제로 겪은 일
리뷰 대응 중 표와 본문의 숫자가 어긋난 적이 있습니다. 원인은 AI가 아니라 **제가 표를 마지막에 고쳤기** 때문이었습니다. 그래서 지금은 원고를 고친 뒤 반드시 **“표·그림·초록·본문의 숫자를 서로 대조하는 표”를** 만들게 합니다. 이 표 하나가 리뷰어의 지적을 여러 개 막습니다.

숫자 대조표 프롬프트

아래를 원고 마지막에 붙이면, 제출 전에 숫자 불일치를 잡을 수 있습니다.

PROMPT

```
원고 본문 · 초록 · 표 · 그림 캡션에 등장하는 모든 숫자를 추출해
"숫자 / 나오는 위치 / 표의 값 / 일치 여부" 표로 만들어줘.
일치하지 않는 항목은 HIGH로 표시하고, 어느 쪽이 맞는지 원자료로 확인해.
확인 못 하면 "원자료 확인 필요"라고 적어.
```

10 / PROMPTS

복사해서 쓰는 프롬프트 15개

괄호 안만 바꿔 쓰세요. 그대로 붙여넣으면 됩니다.

좋은 프롬프트는 길지 않습니다. **역할 · 입력 · 규칙 · 출력 형식 · 멈춤 조건**이 있으면 충분합니다. 아래 15개는 그 다섯 칸을 채운 것입니다.

01 · 연구 질문 다듬기

PROMPT

```
@academic-research-writing
내 연구 아이디어를 PICO로 정리해줘. 설계 후보 2개를 장단점과 함께.
아직 원고는 쓰지 말고, 확인 질문이 있으면 먼저 물어봐.
아이디어: (한 문단)
```

02 · 코드북에서 변수 찾기

PROMPT

```
01_data/codebook의 코드북을 읽고, (결과 변수)와 (노출 변수)에
쓸 수 있는 변수를 "변수명 / 정의 / 척도 / 연도 / 결측" 표로 정리해.
코드북에 없으면 추측하지 말고 "없음"이라고 써.
```

03 · 분석 계획 검토

PROMPT

```
02_analysis/plan.md를 STROBE에 비추어 검토해.
빠진 항목 · 애매한 표현 · 심사에서 문제될 부분을 위험도와 함께.
수정 문장을 제안하되 파일은 아직 고치지 마.
```

04 · 복합표본 분석 코드

PROMPT

(R/Stata)로 분석 코드를 작성해. 조건:
- 복합표본설계(층 · 집락 · 가중치) 반영, 변수명은 코드북 확인 후 주석
- 원자료 수정 금지, 결측 처리 명시
- 코드 먼저 보여주고 승인 후 실행
- logs/에 패키지 버전과 실행 시간 기록

05 · 표 1 만들기

PROMPT

(노출/결과)에 따라 표 1을 만들어줘.
연속변수는 평균(SE) 또는 중앙값(IQR), 범주형은 가중 %.
결측 수를 각 변수에 표시하고, 전체 n과 분석 n을 구분해.
03_output/tables/table1.csv와 .docx로 저장해.

06 · 결과 섹션

PROMPT

표 1~3만 근거로 Results를 써줘.
문장당 숫자 최대 하나, 나머지는 표를 가리켜.
표에 없는 숫자는 만들지 말고, 없으면 "표에 없음"이라고 써.
한국어 본문 + 영어 초록 3문장.

07 · 고찰의 한계

PROMPT

이 연구의 한계를 심사자가 지적할 순서로 7개 나열해.
각 한계에 "왜 문제인가 / 어떻게 완화했나 / 남은 위험"을 붙여.
과장하지 말고, 데이터가 지지하지 않는 방어는 쓰지 마.

08 · 인용 검증

PROMPT

참고문헌을 하나씩 검증해: 존재(DOI/PMID), 주장 지지(쪽 · 표),
접근 수준(full_text/abstract/metadata_only).
확인 못 한 것은 "확인 불가"로 표시하고 사용하지 말라고 해.

09 · 자체 심사

PROMPT

```
@paper-review
원고 · 표 · 그림 · 덱을 심사자 관점에서 감사해.
HIGH/MED/LOW로 나누고, 각 항목에 파일 · 줄 · 수정안 3줄.
고치지는 말고 지적만 해.
```

10 · 숫자 대조

PROMPT

```
본문 · 초록 · 표 · 캡션의 모든 숫자를 "숫자/위치/표 값/일치" 표로.
불일치는 HIGH, 원자료 확인 필요로 표시해.
```

11 · 발표 아웃라인

PROMPT

```
@deck-craft
(8분/15분) 발표의 슬라이드 제목만 13개. 각 제목은 주장 문장.
청중은 (학회/전공의/검진센터). 본문 · 디자인은 아직 만들지 마.
```

12 · 덱 생성

PROMPT

```
승인한 아웃라인으로 16:9 HTML 덱을 만들어.
1920×1080, 오프라인, 숫자는 표 · 도표에, 도표는 SVG로.
슬라이드마다 발표자 노트 30-45초.
```

13 · 대본

PROMPT

```
슬라이드별 대본. 총 (8)분, 전환 포함.
"시간/말할 것/넘어가는 한 문장" 형식.
숫자는 필요한 것만 읽고, 어려운 용어는 첫 등장에 풀어.
```

14 · 큐카드

PROMPT

```
@script-to-cuecards
05_talk/script.md로 A4 2분할 큐카드 PDF.
발표 파트와 Q&A 파트를 따로. 인쇄는 단면 100%.
```

15 · 인계(handoff)

PROMPT

```
지금까지의 결정 · 파일 경로 · 다음 할 일 · 주의점을
notes/handoff.md로 정리해. 새 세션에서 이 파일만 읽고 이어갈 수 있게.
확인 필요 항목은 따로 표시해.
```

---

11 / ROADMAP

8주 로드맵,
그리고 오늘 저녁 세 가지

완주가 목표가 아닙니다. 첫 표 하나가 목표입니다.

한 편의 논문을 처음부터 끝까지 혼자 해내려면 보통 6~12개월이 걸립니다. AI를 써도 **연구 자체가 빨라지지는 않습니다.** 빨라지는 것은 **문서·표·발표를 만드는 시간**입니다. 그 시간을 8주로 나눠 봤습니다.

8주 — 논문 한 편 + 학회 발표 하나

주	목표	AI에게 맡기는 것	내가 확인하는 것
1	질문·설계·폴더	PICO, 설계 후보, STROBE 체크	질문이 한 문장인가
2	자료·코드북·문헌	변수 표, 문헌 요약, 인용 장부	코드북 버전·변수명
3	분석 계획 고정	계획 검토, 빈 표 설계	가중치·결측·보정변수
4	주 분석·표 1	코드, 표, 로그	n 일치, 가중 % 합
5	원고 초안	IMRaD, 숫자 대조표	표에 없는 숫자 0
6	자체 심사·수정	paper-review, 인용 검증	HIGH 항목 처리
7	투고 패키지	체크리스트, cover letter	맹검·형식·공개 문구
8	발표 패키지	덱·대본·큐카드	시간 ±10%, Q&A 5개

오늘 저녁에 할 세 가지

- [ ] **OpenCode를 설치하고, 폴더 하나를 연다.** 논문 폴더가 아니어도 됩니다. 연습용 폴더면 충분합니다.
- [ ] **스킬을 복사하고, `@academic-research-writing` 을 불러 본다.** 자동완성에 뜨는지 확인.
- [ ] **내 논문 초록을 붙여넣고 “문제 3개만 짚어줘”라고 한다.** 고치지 말고 지적만 받습니다.

첫 주에 하지 말 것

- 원본 데이터를 AI에게 “알아서 정리”시키기
- 논문 전체를 한 번에 쓰게 하기
- AI가 만든 참고문헌을 그대로 넣기
- 결과가 안 나왔는데 “유의하게” 만들 방법을 묻기
- 환자 정보가 든 파일을 채팅에 올리기

자주 막히는 곳 — FAQ

스킬이 자동완성에 안 보입니다

경로를 확인하세요: `~/.config/opencode/skills/<스킬 이름>/SKILL.md`. 폴더가 한 겹 더 들어가 있으면 (`skills/design-craft/design-craft/SKILL.md`) 인식되지 않습니다. OpenCode를 완전히 종료했다가 다시 켜세요.

AI가 파일을 못 찾습니다

현재 폴더가 맞는지 확인하세요. "파일이 없다"고 하면 **"폴더 목록을 보여줘"**라고 먼저 물어보세요. 그래도 없으면 경로를 직접 알려 주세요.

한국어가 깨지거나 영어로 답합니다

한국어로 쓰라고 명시하세요: "본문은 한국어, 표 머리글도 한국어. 고유명사만 영어." 폰트 문제로 PDF가 깨지면 HTML을 PDF로 인쇄할 때 **한글 폰트가 설치된 PC**에서 하세요.

표가 Word에서 깨집니다

docx는 환경에 따라 레이아웃이 달라집니다. 제출 직전에는 **Word에서 직접 열어** 표 폭·페이지 나눔을 확인하세요. 이 확인은 AI가 대신할 수 없습니다.

비용이 걱정됩니다

긴 세션은 비쌉니다. ① 한 세션에서 한 가지 일만 ② 파일 전체를 붙이지 말고 필요한 부분만 ③ 2시간마다 handoff 후 새 세션. 이 세 가지로 비용이 크게 줄어듭니다.

발표장에서 파일이 안 열립니다

HTML은 발표 PC의 브라우저·폰트에 따라 달라집니다. **PDF를 함께 준비**하고, PDF로 발표하세요. 동영상·애니메이션이 있으면 MP4로 따로 저장합니다.

이 가이드를 업데이트하는 방법

저장소를 fork하거나, 여러분 기관에 맞게 `skills/`를 고쳐 쓰세요. 좋은 프롬프트가 생기면 `guide/prompts.md`에 추가하고 pull request를 보내면 다음 사람이 혜택을 봅니다.

만든 사람

김현수 · 가정의학과
2026 대한가정의학회 추계학술대회
발표 · 수상

이 저장소에 들어 있는 것

스킬 7종 (`skills/`)

이 안내서 (`guide/`)

발표자료·대본·큐카드 (`deck/`, `script/`, `cuecards/`)

설치 스크립트 (`install/`)

라이선스

글·그림: CC BY 4.0

스킬 코드: 각 스킬의 LICENSE

인용한 논문: 원저작자

이 문서는 교육 목적의 안내서이며, 특정 환자의 진단·치료를 대신하지 않습니다. 인용한 수치와 연구 결과는 원문을 확인하세요. AI가 만든 초안은 반드시 사람이 검토·서명해야 합니다.